

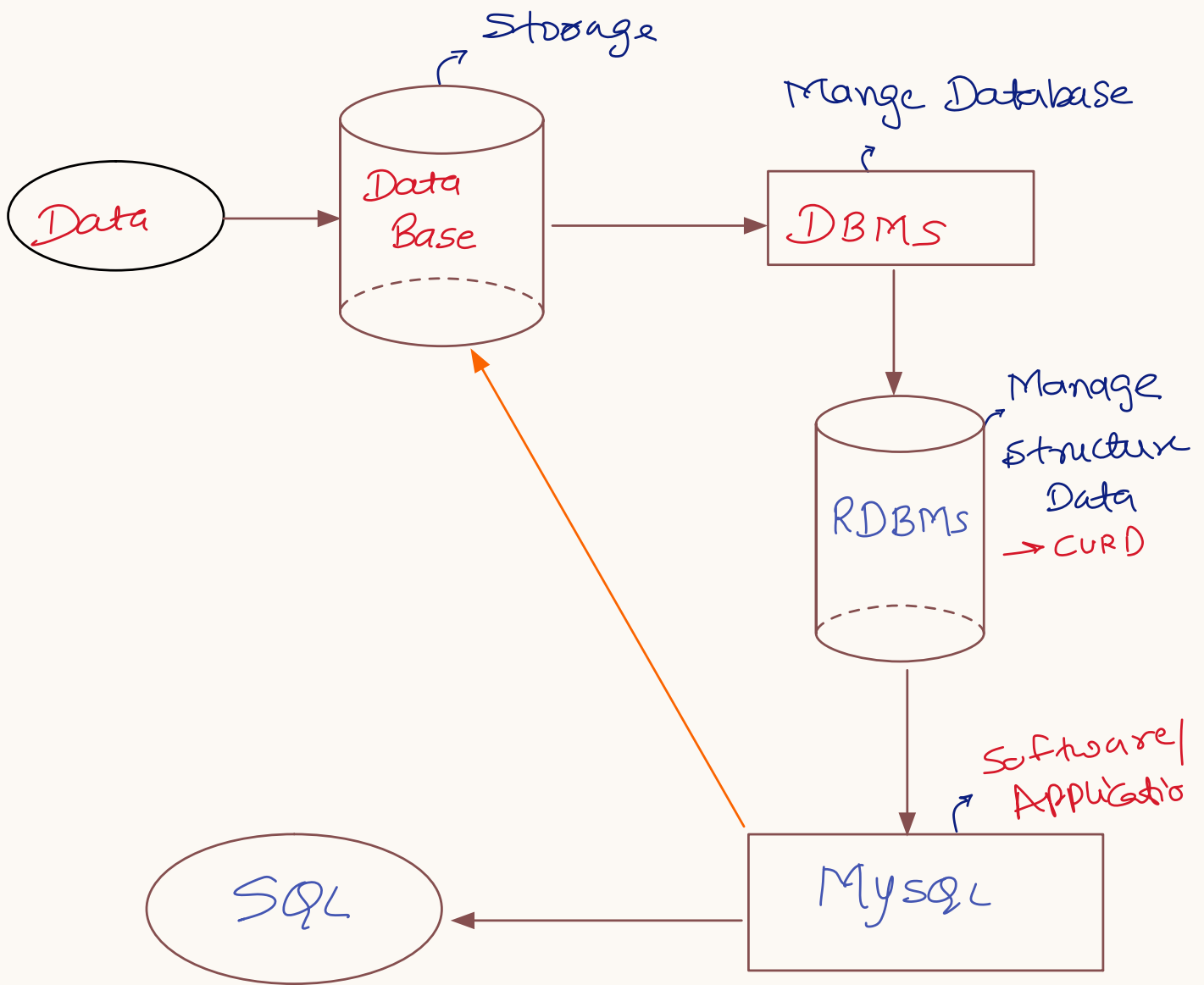
MMY

SOL

By -

Deepraj..

SQL



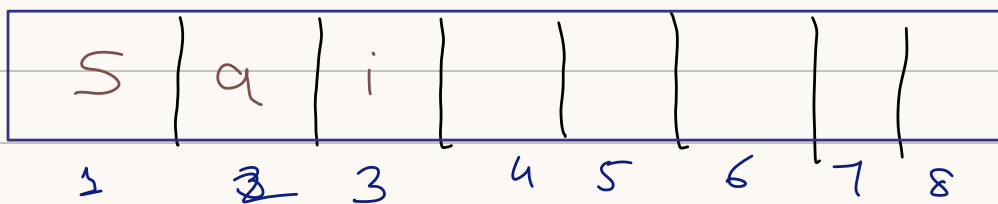
↳ It is a scripted programming lang which helps us to communicate with RDB

id	Name	Email	Phone NO	DoB	Gender
101	A	-	9 ..	"2000-01-12"	M
102	B	-	8-	-	F
103	C	-	7-	-	M
					F

Datatypes

- ① Numeric DT → int, Smallint, Bigint, Tinyint
- ② String/Character DT → Char, varchar, Text
- ③ Date DT → Date, Datetime, Year
- ④ Boolean

Char(8)



10 5, 50 55

Numeric DT

<u>Data Type</u>	<u>Storage</u>	<u>When to use</u>
Int	4 Bytes	Whole number (age, salary, roll no)
Small int	2 Bytes	Small number (age, wt, ht)
Big int	8 Bytes	High number (Aadhar, pan, tr, Bank)
Decimal	depends on Precision	Salary, UPI, distance, Height, w
float	4 Bytes	
Double	8 Bytes	

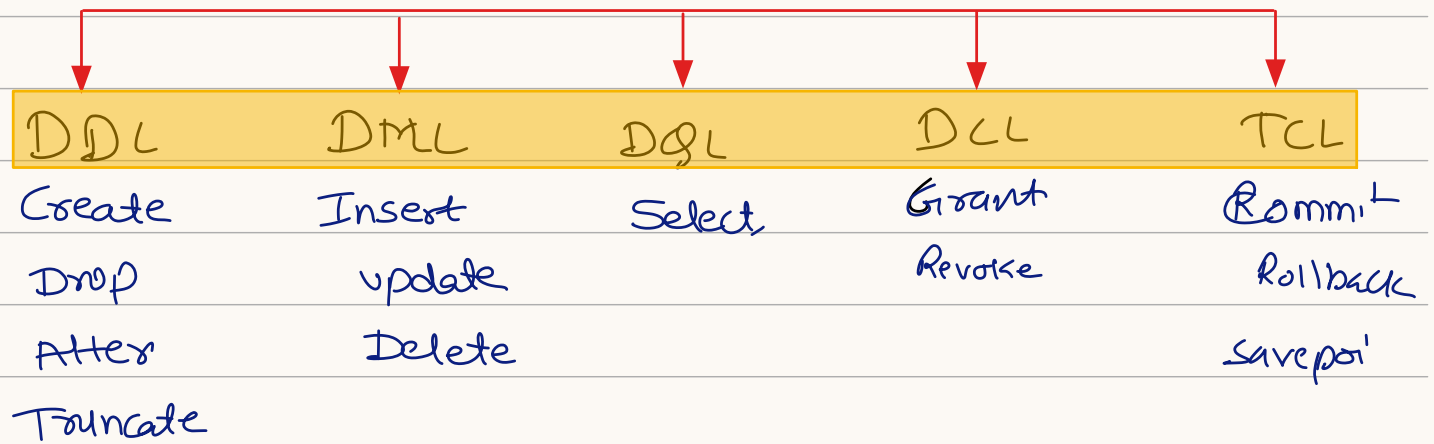
String / character DT

<u>Data type</u>	<u>Storage</u>	<u>When to use</u>
Char(n)	Fixed Bytes	Gender, pincode,
Varchar(n)	variable length	Name, place name, Email.
Text	upto 65535	Add, description

Date & Time DT

<u>Datatype</u>	<u>Storage</u>	<u>When to u</u>
Date	3 Bytes	"2025-11-27"
Time	3 Bytes	3:54:00"
Datetime	8 Bytes	"2025-11-11": 3:54:00
Timestamp	4 Bytes	Current time
Year	1 byte	2025

MySQL Commands



DDL	DML	DQL	DCL	TCL
Create	Insert	Select	Grant	Commit
Drop	update		Revoke	Rollback
Alter	Delete			Savepoint
Truncate				

DDL

Data Definition Language

Use to make structure of Database or change the structure of DB

(1) Create:- Create a Database [Table]

Syntax:- Create Database DatabaseName;
Create Database studentdb;

Create table

Syntax:- Create table tablename (col1, col2, col3);
Create table Students (id int, Name varchar(10),
age int, DOB Date);

Alter:- Modify the structure of table.

- Add a new column
- Modify the Column name
- Modify the Datatype / column structure

① Add a column

Syntax:- Alter table tablename add column dt;

Alter table students add city varchar(30);

Syntax:- Alter table tablename add phoneNo int,
add LastName varchar(20);

② Modify the Column

Syntax:- Alter table tablename Modify Col; w/z

Alter table students modify id enrollment_id;

Drop:- Delete a Entire table / databases

Syntax:- Alter table table-name Drop column
Column-name;

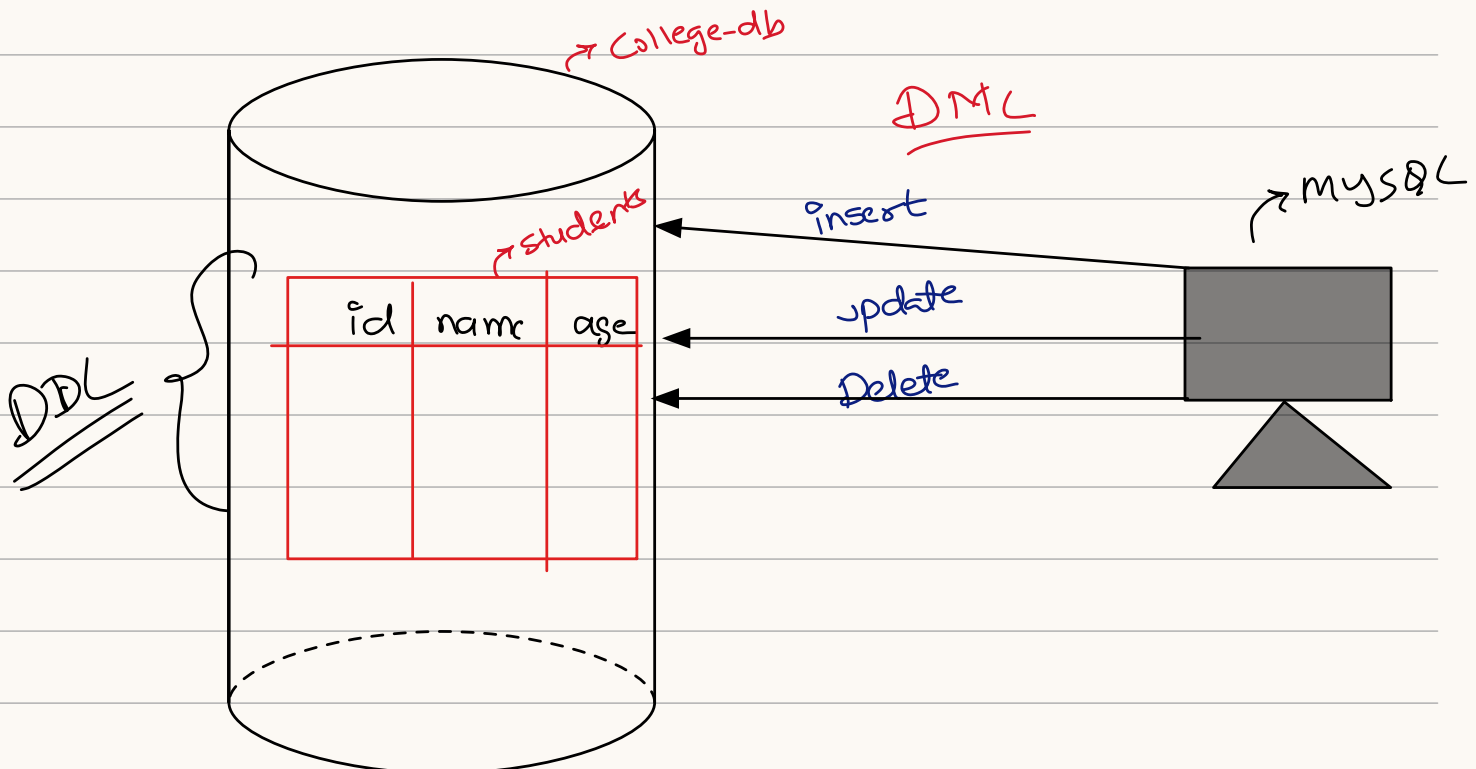
Truncate:-

It will delete all the Rows from the table but Structure will there

→ Truncate is faster than Delete

Syntax:- Truncate table table name ;

Truncate table students ;



DML

Data manipulation language

→ add a data → insert

→ update a data → update

→ Delete a certain Row → Delete

① Insert:- Insert a Data in table

Add a New Data in table.

There are two ways to Insert Data

① Explicit Insert

② Implicit Insert

Explicit Insert:-

Syntax: Insert into tablename (col1, col2)
values (val1, val2, - .);

Insert into students (id, name, age, Gender, phone-no)
values (101, 'vivek', 25, 'Male', 942108890);

Implicit insert → No need mention column name
value must be exact with column name

Syntax:- Insert into tablename

values (value1, value2,) .

Insert into students

```
values (102, "Deepraj", 25, "Male", '19421025801'),  
values (103, "Karthik", 26, "Male", '17219272681'),  
values (104, "Sivisha", 23, "Female", '8192678010');
```

Update - Modify the existing Data

Syntax:- update tablename
Set column = value
Where condition ;

```
update student  
set age = 30  
where id = 103 ;
```

Delete:- Used to remove Row from table

Syntax:- Delete from tablename
Where condition ;

```
Delete from Students  
Where id = 103 ;
```

Commands	Removes	Keep structure	Can we use where
Delete	Selected Rows	yes	yes
Truncate	All Rows	yes	no
Drop	Table + Rows	No	No

DQL

Select → To Retrieve data from table.

To Access data from table

Constraints

Imagine you are joined a College admission team
Before admitting Student College Must follow certain
Rules. These Rules ensure Clean, correct & Meaningful data in
the Database

Constraints

Constraints are Rules Applied to column in a
table

This will make sure our database is accurate,
Reliable, meaningful & Data Redudency.

Admission Database

- ① Departments
- ② students.

Constraints in SQL

- Not NULL
- unique
- Check
- Default
- Primary Key
- foreign Key
- Composite Key
- Auto increment

Not NULL → Record should be Not empty
Not NULL

→ Mandatory field / column

Unique! We cannot have duplicate value
NULL values can be allowed

Check:- Select provided values it self

Default:- Setting some default value.

Primary Key: A primary key uniquely identify each
Row

It is a combine of two Rule

↳ Unique
+
↳ Not null

Unique identification

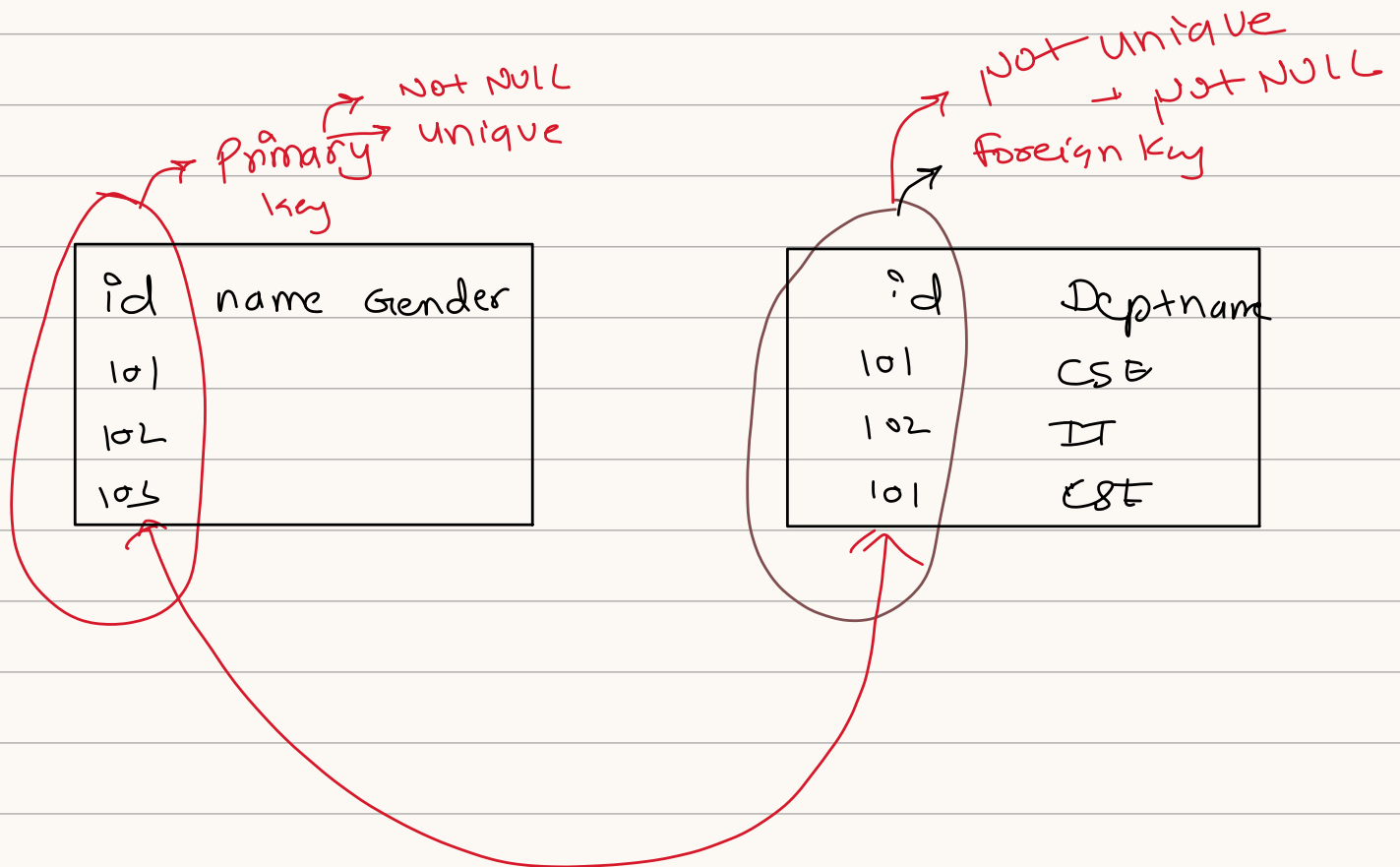
Data integrity

Relationship

for each table we have one primary key.

Foreign Key → It is key Help us to locate Relationship b/w two table

foreign key is a Reference of primary key
→ Not unique + not NULL



Admission Database.

→ Departments

→ students

Department table

Department id must be unique & Not Null

Each department needs its own identity

No duplicates & empty id.

→ Primary Key

Department Name

Avoid wrong or inconsistent names like CSEEE

→ Check

Students

Student id

Every student must have their own id no.

Two students cannot share the same ID & ID cannot be empty

→ Unique & Not NULL

↳ Primary Key.

Name: -

College cannot admit a student without a name

→ Not NULL

Gender should be Male or Female

Avoid wrong entry like "mail", "female", 123 etc.

→ Check constraint

Email must be Unique + Not NULL

Email should be unique

→ Unique + Not NULL

Fees paid cannot be Negative (Default=0)

A student can't pay Negative fees

Some may not have paid yet

→ $\text{check}(\text{fees-paid} \geq 0)$

department_id	department_name
1	Computer Science
2	Information Technology
3	Electronics
4	Mechanical
5	Civil

students_id	name	gender	dob	email	phone_number	total_fees	fees_paid	city	admission_year	scholarship_status	department_id
1	Amit Sharma	Male	2002-05-14	amit.sharma@example.com	9876543210	50000.00	20000.00	Hyderabad	2021	No	1
2	Priya Verma	Female	2001-11-22	priya.verma@example.com	9812345678	50000.00	30000.00	Bangalore	2020	Yes	2
3	Rohan Gupta	Male	2003-03-10	rohan.gupta@example.com	9823456712	48000.00	25000.00	Mumbai	2022	No	1
4	Sneha Reddy	Female	2002-07-18	sneha.reddy@example.com	9900123456	52000.00	40000.00	Hyderabad	2021	Yes	3
5	Arjun Mehta	Male	2001-09-30	arjun.mehta@example.com	9876501234	50000.00	50000.00	Delhi	2023	No	2

Clause

→ It is a type of instructions given to the table to get some data.

Types of clauses

- select
- from
- where
- Groupby
- Having
- orderby
- Joins
- limit
- offset

Select → Retrieve the data from tables

→ Access the data from table

Syntax -

Select col1, col2, ... from table-name,



we are retrieving certain columns.

Select * from tablename;

↳ Retrieving all data

↳ Rows x columns

Where! Filters a Data based on Condition.

Where Operators

Comparison Operators	Logical Operators	Range Operators	Membership Operators	Search Operators
=	And	Between	in	Like
!=, <, >	or		not in	
>, >=	not			
<, <=				

Comparative operator

Students

student_id	Name	Gender	Total Fees
1	Amit Sharma	Male	50000
2	Priya Verma	Female	50000
3	Rohan Gupta	Male	48000
4	Sneha Reddy	Female	52000
5	Arjun Mehta	Male	50000
6	Kavya Nair	Female	45000

Retrieve a Rows whose total fees greater than 50000

Total fees > 50000

Total fees >= 50000

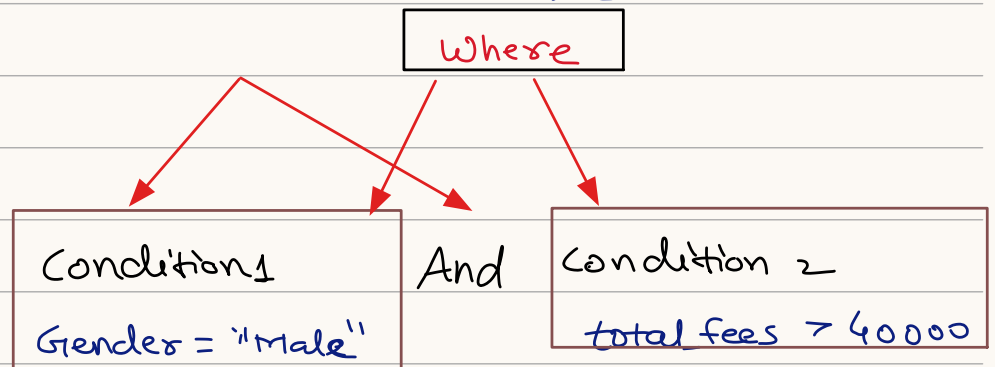
Select * from Students

Where total-fees >= 50,000 !

↓
>
<
<=
==

Logical operators

And → All Condition must be True



True True

student_id	Name	Gender	Total Fees
1	Amit Sharma	Male	50000
2	Priya Verma	Female	50000
3	Rohan Gupta	Male	48000
4	Sneha Reddy	Female	52000
5	Arjun Mehta	Male	50000
6	Kavya Nair	Female	45000

Amit male 50000

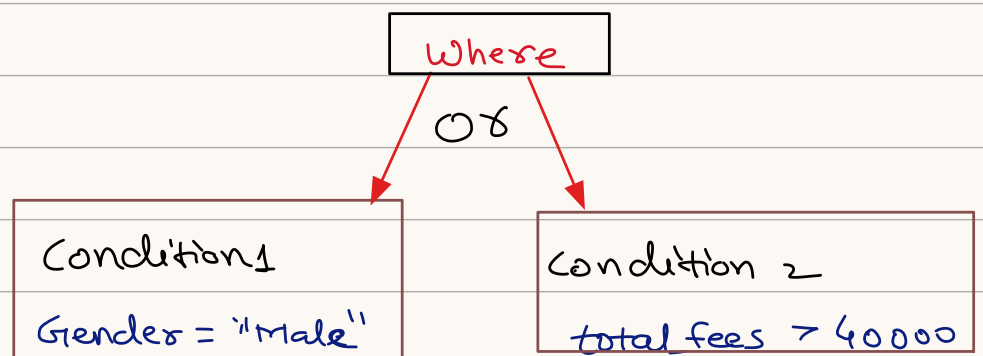
X

X

X

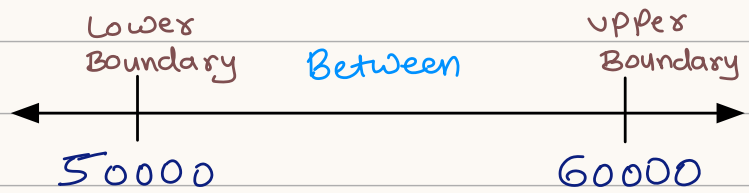
✓

OR → At least one condition must be True



student_id	Name	Gender	Total Fees
1	Amit Sharma	Male	50000
2	Priya Verma	Female	50000
3	Rohan Gupta	Male	48000
4	Sneha Reddy	Female	52000
5	Arjun Mehta	Male	50000
6	Kavya Nair	Female	45000

Between → check if values is within a Range



student_id	Name	Gender	Total Fees
1	Amit Sharma	Male	50000
2	Priya Verma	Female	50000
3	Rohan Gupta	Male	48000
4	Sneha Reddy	Female	52000
5	Arjun Mehta	Male	50000
6	Kavya Nair	Female	45000

✓
✓
✗
✓
✓
✗

In → check if value exists in a list

Not In → check if value doesn't exist in a list

student_id	Name	Gender	Total Fees
1	Amit Sharma	Male	50000
2	Priya Verma	Female	50000
3	Rohan Gupta	Male	48000
4	Sneha Reddy	Female	52000
5	Arjun Mehta	Male	50000
6	Kavya Nair	Female	45000

total fees in (50000, 45000)

Select * from Student

where total fees = 50000 ;

Select * from Students

where total fees = 45000 ;

Select * from students

where total fees in (50000, 45000)

Like' It is used to search for patterns in text column

It is used with wildcards

% → matches any No of characters (0 or more)

student id	Name	Gender	Total Fees	"A%"	"%A"	"%.A%"
1	Amit Sharma	Male	50000	✓	✓	✓
2	Priya Verma	Female	50000	✗	✓	✓
3	Rohan Gupta	Male	48000	✗	✓	✓
4	Sneha Reddy	Female	52000	✗	✗	✓
5	Arjun Mehta	Male	50000	✓	✓	✓
6	Kavya Nair	Female	45000	✗	✗	✗

Orderby

student_id	Name	Gender	Total Fees
1	Amit Sharma	Male	50000
2	Priya Verma	Female	50000
3	Rohan Gupta	Male	48000
4	Sneha Reddy	Female	52000
5	Arjun Mehta	Male	50000
6	Kavya Nair	Female	45000

Select * from Tablename
order by tablename Asc/Dsc;

Select * from Students
Order by Students;

Groupby

Groupby works with 3 steps

student_id	Name	Gender	Total Fees
1	Amit Sharma	Male	50000
2	Priya Verma	Female	50000
3	Rohan Gupta	Male	48000
4	Sneha Reddy	Female	52000
5	Arjun Mehta	Male	50000
6	Kavya Nair	Female	45000

① Split

② Apply

↳ Aggregated function
→ min, max

③ Combine

→ sum, avg

Select Gender, Avg (Total fees) as Avgfees
from Students

Group by Gender ;

Order of writing SQL Query:

Select
Distinct
From
Join
Where
Groupby
Having
order by
Limit
offset

order of Executing SQL query

From
Join
Where
Groupby
Having
Select
Distinct
'Order by
Limit
offset

SQL functions

SQL contain Built in operators in the SQL function

- ① Single (Scalar) funⁿ
- ② Groupby (Agg) funⁿ

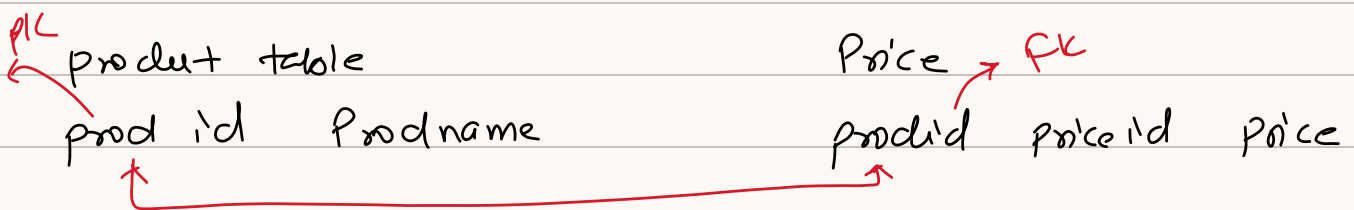
What happens when we store data in
on table or worksheet

- Data Duplicate (Data Redundancy)
- Update Anomaly
- Delete Anomaly
- Insert Anomaly

To solve above problem we depend on
Joins & Relationship

Relationship: -

Connecting a tables together using keys
↳ PK
↳ FK



Types of Relationship: -

- ① One to one Relationship
- ② One to many Relationship
- ③ Many to many Relationship
- ④ Many to one Relationship

① One to one Relationship

When table A Record/Row is connecting with Record/Row in table B is called one to one Relationship

① user → Aadhar number / Pan./Aec No

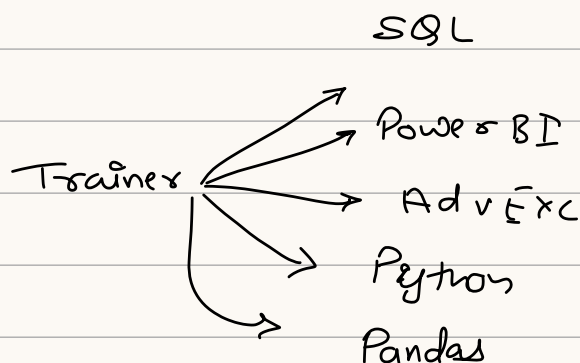
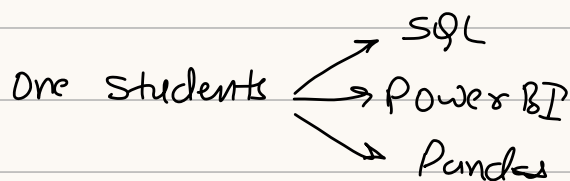
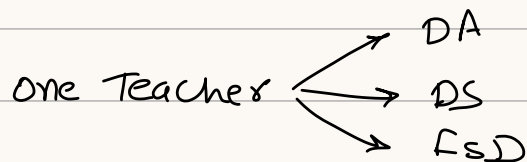
② Student → Enrollment number

③ Area wise → pincode

Kukatpally → 500085

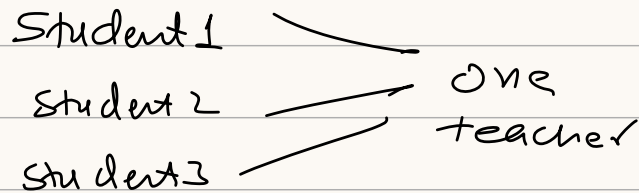
② one to many Relationship

When one Record/Row in table A is getting connected with multiple Records/Rows in table B



Many to one Relationship -

When Multiple Records/Rows getting connected with one Record/Row in Table B.



Many to many

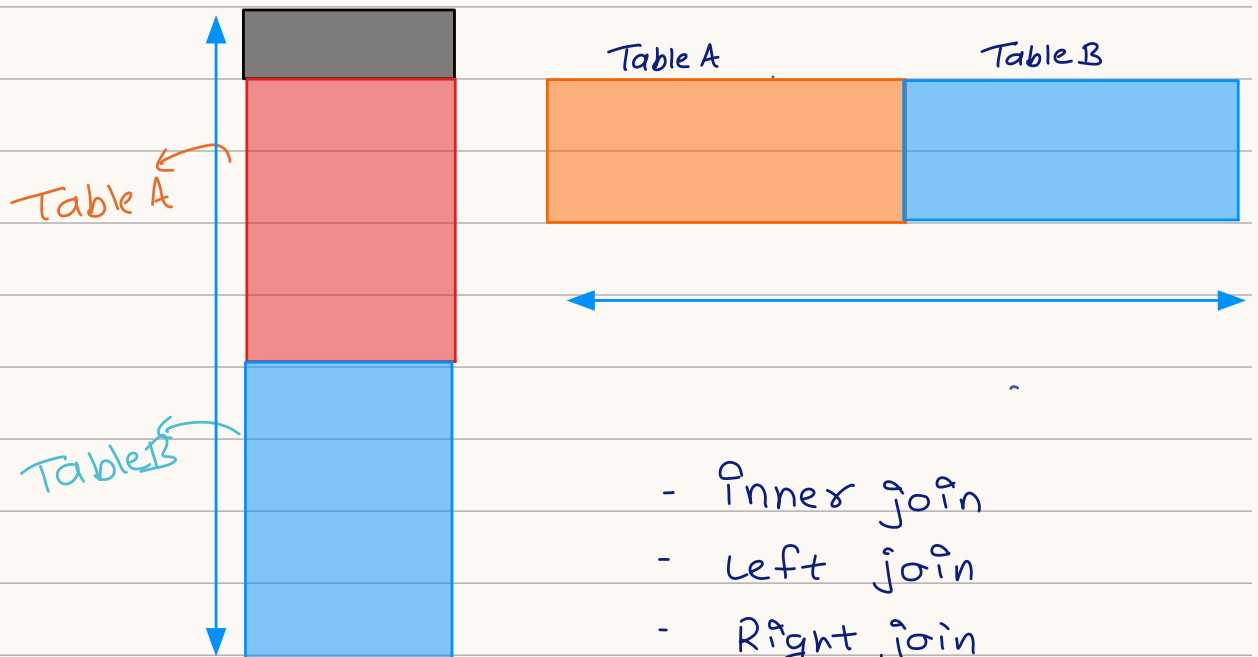
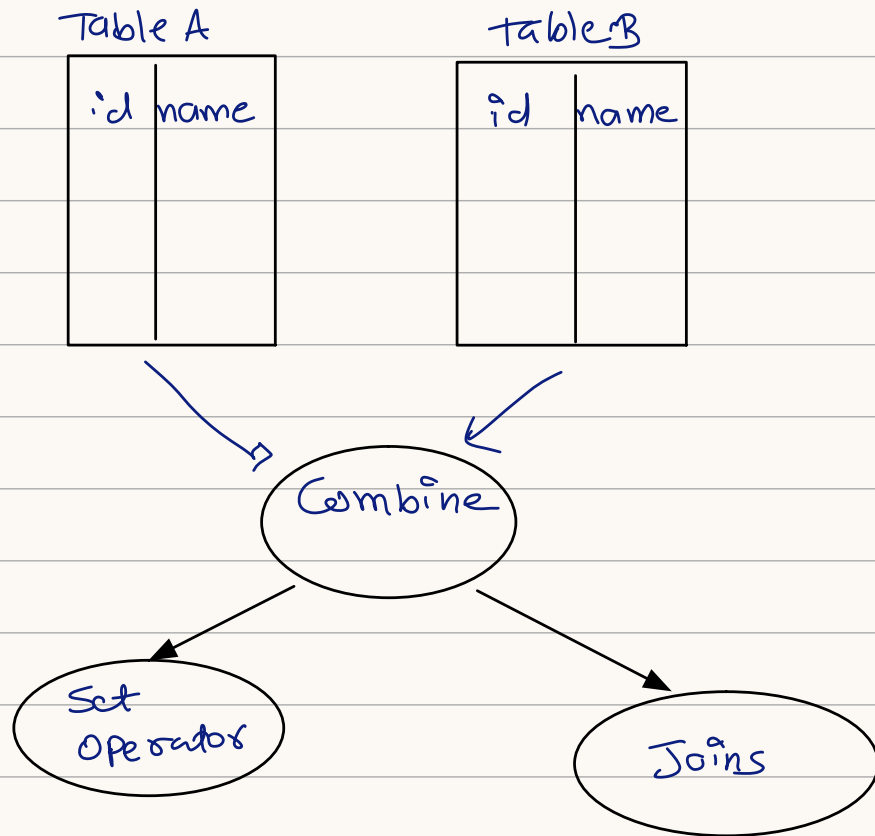
When Many Records or Rows in table A is getting connected with many Records/Rows in table B

Students $\longleftrightarrow$ Course

Students $\longleftrightarrow$ Subject

Joins

→ When we are connecting one more table together based on pk & fk with a Relationship is called Joins.

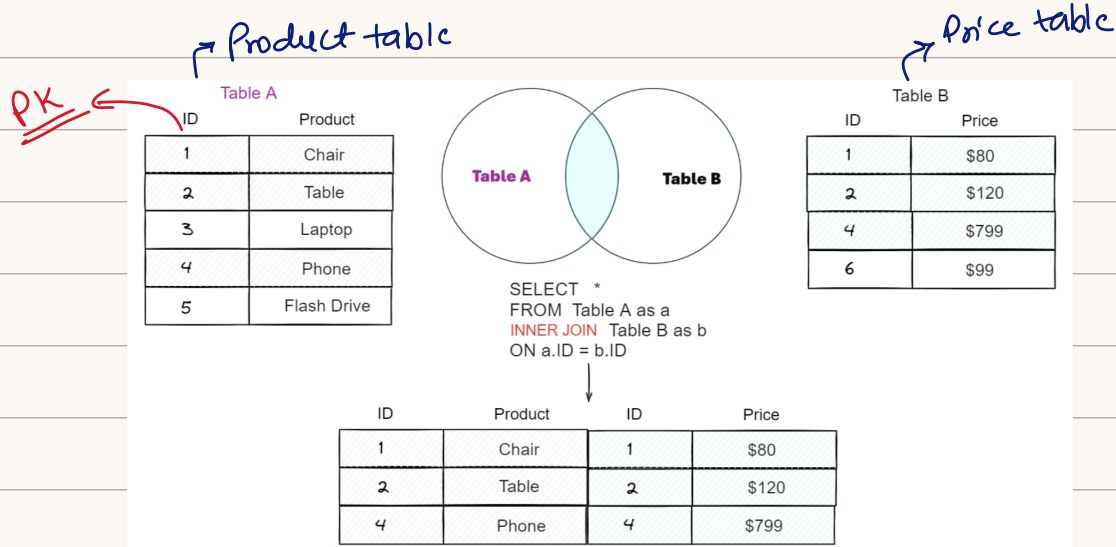


- Union
- Union All

- Inner join
- Left join
- Right join
- Self join
- Cross join

Joins

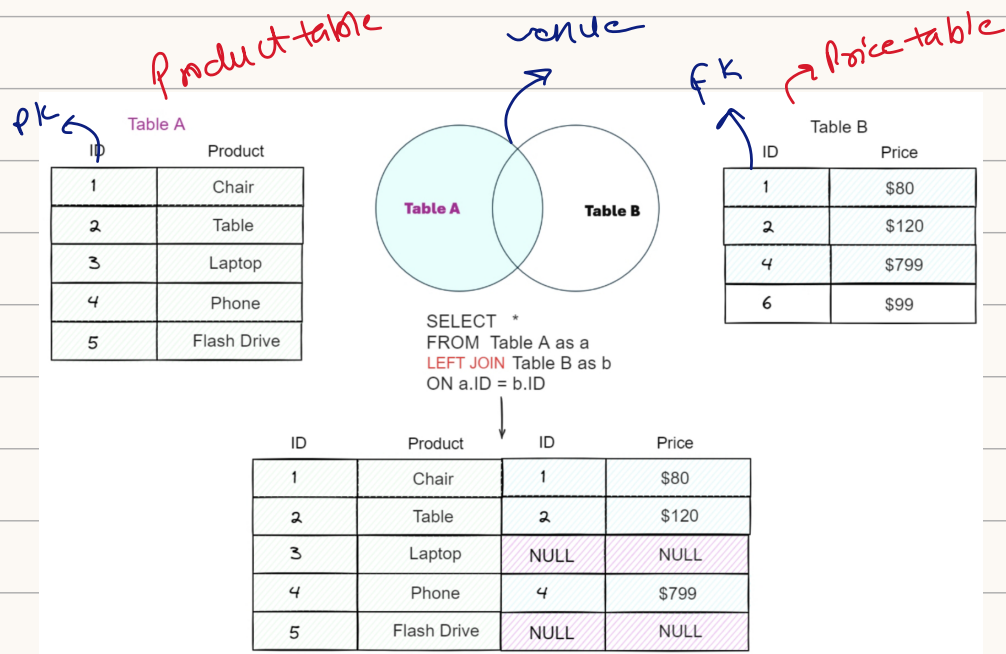
Inner Join: - Return only Rows / Records with Matching values in Both the table



Syntax! select * from products as p
inner join / join prices as pr
on p.id = pr.id ;

select p.Product, pr.Price from products as p
inner join prices as pr
on p.id = pr.id ;

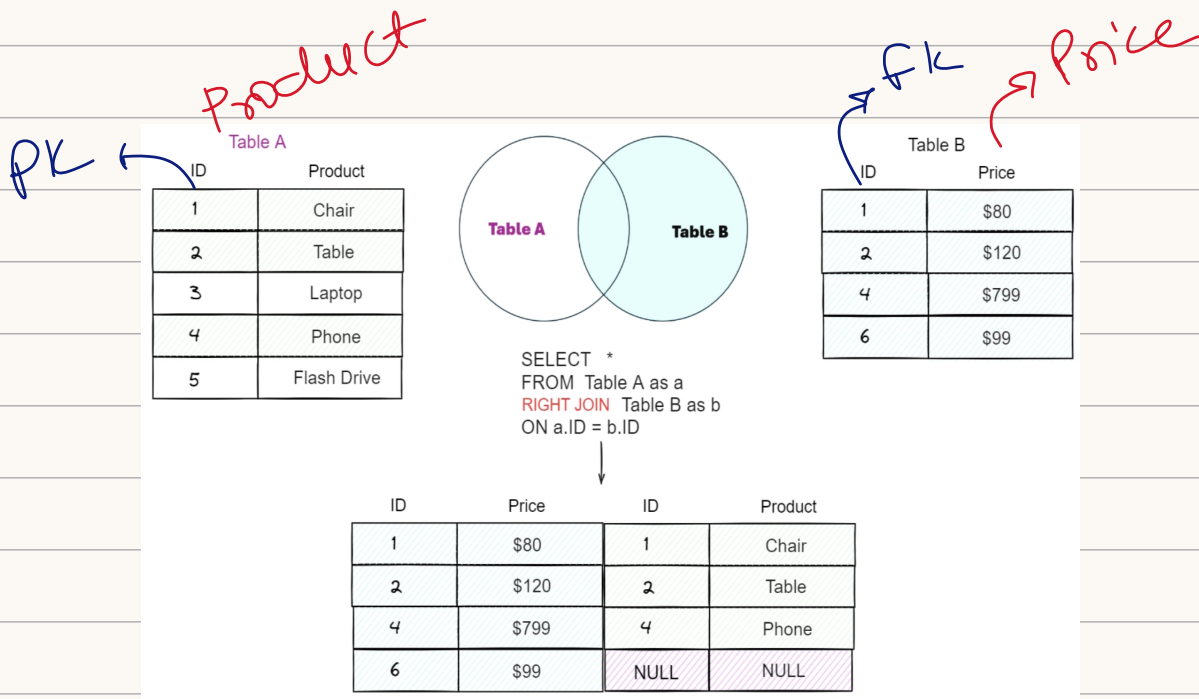
Left Join. - Return Rows from Left table and Matching Rows from Right table. Unmatched Rows from table show as **NULL**



Select * from Products AS p
 Left join Prices as pr
 on p.productid = pr.productid;

select p.productname, pr.price from products as p
 Left join Prices as pr
 on p.productid = pr.productid;

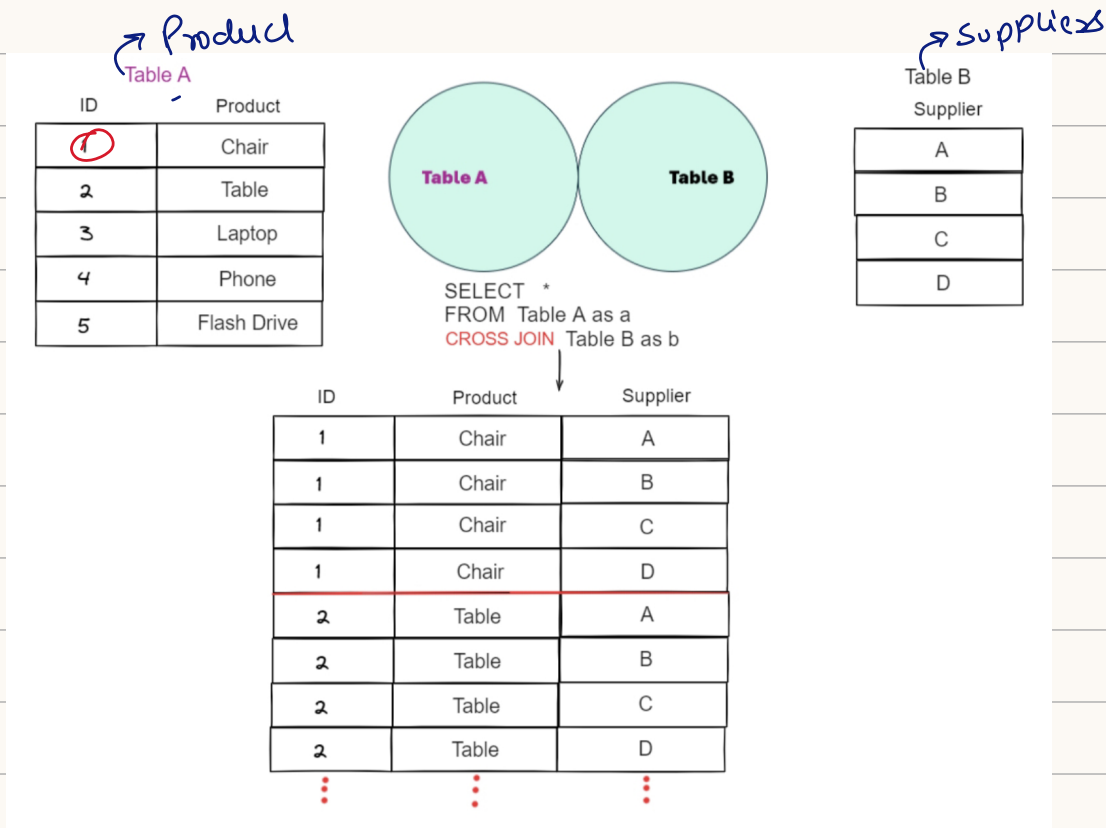
Right Join :- Return all Rows from Right table and Matched Rows from Left table Unmatched Rows from the Left show as **NULL**



Select * from products as p
Right Join prices as pr
on p.id = pr.id ;

Cross Join

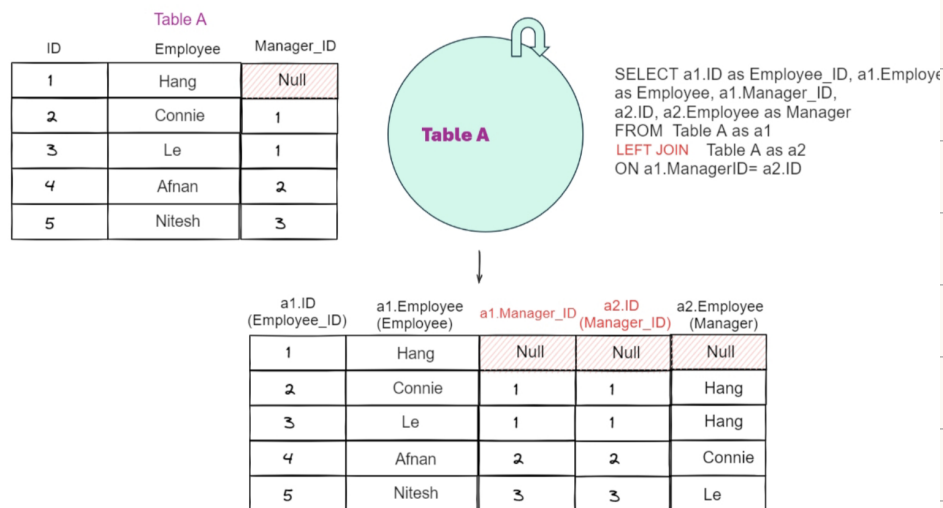
Return the Cartesian product Every combination of Rows from both table



Self Join

A Self Join is a Regular Join where table is joined with itself.

It is useful for comparison rows within the same table



Set operators

Set operators are used to combine the results of two or more select queries

Types of set operators

① union

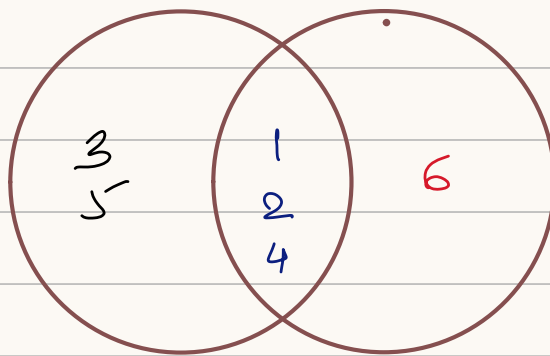
② union all

① union:-

union in SQL combine the results of two or more select statements in single result set, removing duplicate rows to ensure that each row is unique.

→ Product

ID	Product
1	Chair
2	Table
3	Laptop
4	Phone
5	Flash Drive



→ Price table

ID	Price
1	\$80
2	\$120
4	\$799
6	\$99

union



1
2
3
4
5
6

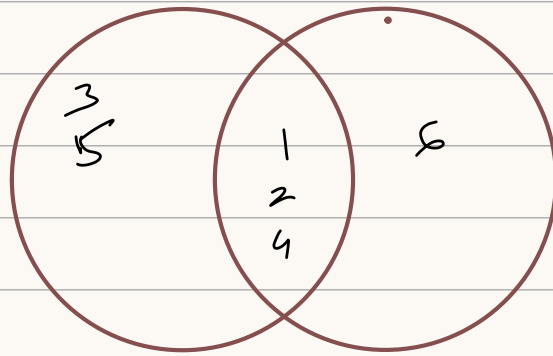
Query:- select id from products as p
union

select id from prices as p2

Union all

Union all in SQL combines the result of two select statement into single result set with duplicate values.

ID	Product
1	Chair
2	Table
3	Laptop
4	Phone
5	Flash Drive



ID	Price
1	\$80
2	\$120
4	\$799
6	\$99

Select id from products as p

union all

Select id from prices as pr

1

2

3

4

5

1

2

4

6

